

# Clase 1

lunes, 6 de abril de 2026 19:11

## Arquitecturas:

Establece un marco de referencia para determinar el alcance del desarrollo.

**Monolítico:** Estilo que consiste en crear una aplicación autosuficiente que contiene en un empaquetado toda la funcionalidad necesaria para realizar la tarea para la que fue diseñada. Es un Todo en uno, pero si tocas algo, tienes que cambiar el resto.

Ventaja: simple de desarrollar en un principio, fácil de testear,  
Desventaja: el trabajo en equipo es complejo,

**Cliente-Servidor:** Hay dos partes donde el servidor le brinda un servicio al cliente. Cliente puede ser delgado, grueso o inteligente lo cual es determinado por el nivel de independencia del cliente. Mientras que el servidor es una pc en la red, lejano al cliente y administra los recursos. Un navegador es considerado un Thin Client, solo muestra lo que le envía el servidor.

Ventajas: centralización del control y los datos, más fácil de mantener y actualizar el servidor, seguridad concentrada en un punto y escalable agregando más servidores.  
Desventajas: servidor caído -> clientes afectados, cuellos de botella, requiere infra de red confiable y latencia depende del servidor.

**Peer-to-peer:** Sirve para compartir recursos, a más usuarios conectados, mejor experiencia. Ej: torrent.

**Arquitectura en capas:** Distribuye las tareas en equipos, Ej: Front, negocios, datos, entidades. Si se cambia la BDD se rompe todo.

**Clean Architecture / Cebolla:** Tipo en capas pero acá tiene un núcleo que son las reglas de negocio y las externas dependen de esta.

**MicroKernel:** Se compone de un Core y alrededor hay plug-ins que desarrollan aplicaciones extensibles o adicionales que añaden características. Ej: Linux o VS Code

**Service-Oriented Architecture:** Software as a Service, se alojan distintos módulos que llevan a cabo soluciones específicas completas. Relaciones entre sistemas y servicios varios. Ej: Aeropuerto y aerolíneas, cada una sabe como manejarse dentro del aeropuerto. Se usa en el banking.

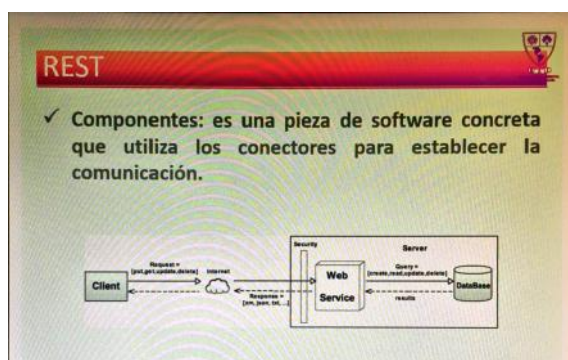
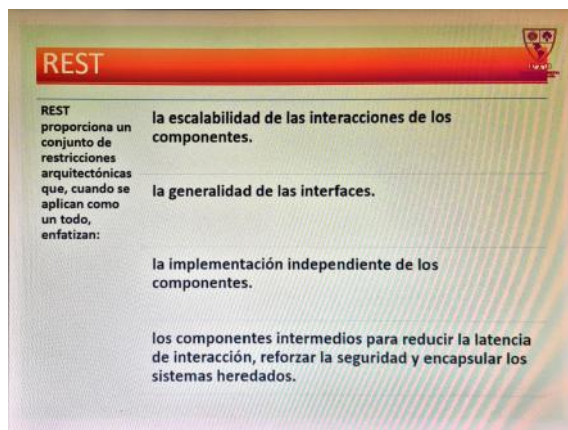
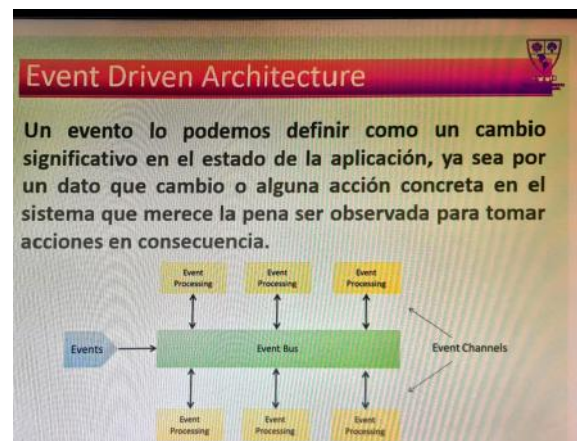
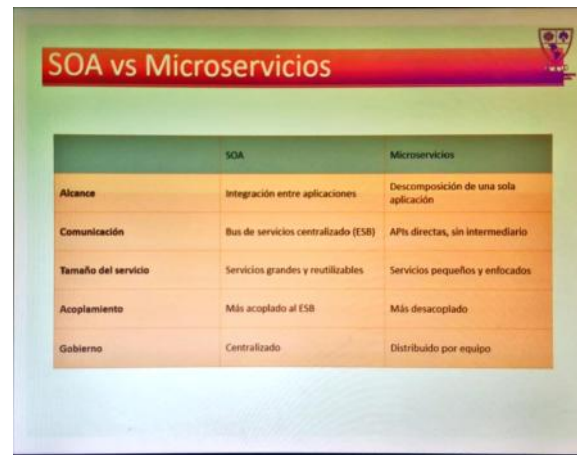
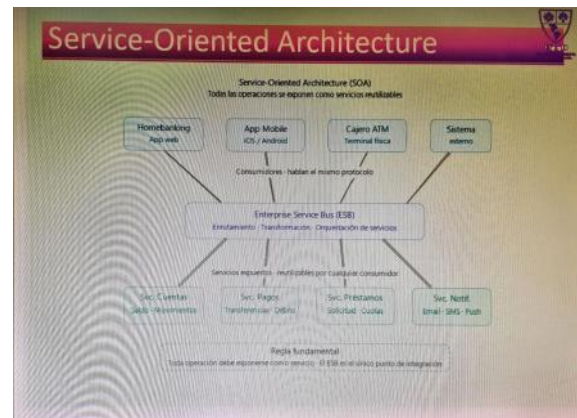
Desventajas: Complejidad alta de integración inicial, bus de servicios puede ser un cuello de botella, requiere estándares y gobernanza estricta y hay un overhead de comunicación(SOAP/XML).

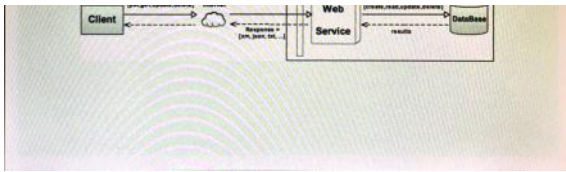
**Microservicios (Tp Grupal):** Consiste en crear pequeños componentes de tipo clases con una sola responsabilidad y es autosuficiente. API Gateway es un pasamanos que recibe peticiones y devuelve las rutas que solo él conoce. Cada microservicio debería manejar su propia BDD y eso lo hace altamente cohesivo.

**Event Drive Architecture:** un bus se encarga de recibir eventos y los direcciona a la unidad de procesamiento correspondiente.

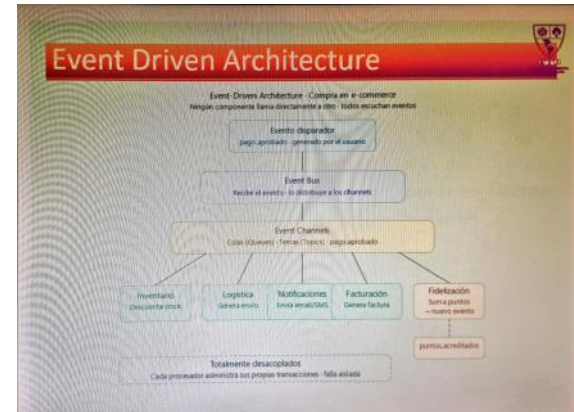
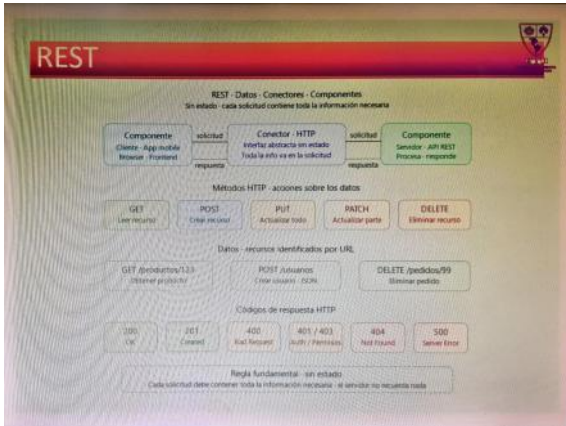
Ventaja: desacoplamiento total.  
Desventaja: Atomicidad difícil de garantizar.

**Representational State Transfer (REST):** conjunto de restricciones que crean un estilo arquitectónico y que es común utilizarse para crear aplicaciones distribuidas. REST no tiene memoria, no guarda datos.





## Verbos de http





<https://caniuse.com/>

Etiqueta de apertura

Etiqueta de cierre

Contenido

Elemento

•**Etiqueta de apertura:** se reconoce porque está al inicio del código y se encuentra entre < > (paréntesis angulares). En este ejemplo son las que encierran la letra p.

•**Etiqueta de cierre:** son similares a la etiqueta de apertura con la diferencia que se encuentra una barra inclinada </ > antes de finalizar la etiqueta.

•**Contenido:** es la información que se encuentra entre la etiqueta de apertura y la etiqueta de cierre.

Etiquetas y atributos

<a href="https://ejemplo.com"> Texto del enlace </a>

Etiqueta de apertura

Atributo

Valor del atributo

Contenido

Etiqueta de cierre

Reglas fundamentales

- Las etiquetas van entre < >
- La mayoría tiene apertura <etiqueta> y cierre </etiqueta>
- Los atributos van siempre en la etiqueta de apertura
- El valor del atributo va entre comillas dobles: atributo="valor"

HTML5

Lo primero que se hace en una página suele ser indicar el conjunto de caracteres que se utiliza. En versiones anteriores de HTML, se hacía usando un elemento muy complejo <meta> .

Ahora es muy simple:

```
<meta charset="UTF-8">
```

<!DOCTYPE html>

<html lang="en">

<head>

<!DOCTYPE html>

<html lang="en">

<head>

<title>My perfect website</title>

<meta charset="utf-8" />

<link rel="preconnect" href="https://s3.mysite.com">

<link rel="preconnect" href="https://www.mysite.com">

Estructura mínima de un documento HTML5

<!DOCTYPE html>

<html>

<head>

<meta charset="UTF-8">

<title>Mi página</title>

</head>

<body>

<!-- contenido -->

</body>

</html>

<!DOCTYPE --> Declara que es HTML5

<html --> Rara del documento

<head --> Metadatos (no visible)

<meta --> Configuración de caracteres

<title --> Título en la pestaña del navegador

<body --> Contenido visible de la página

RENDIMIENTO E INTEGRACIÓN

- Web Workers:** permite delegar la evaluación JavaScript para subprocesos en segundo plano, lo que evita que estas actividades ralenticen eventos interactivos.
- XMLHttpRequest Nivel 2:** permite buscar de forma asíncrona algunas partes de la página, mostrando contenido dinámico, que varía según el tiempo y las acciones del usuario. Esta es la tecnología detrás de Ajax.

Ante errores en el marcado, todos los navegadores deben comportarse de la misma forma.

PDF

clase 2  
evaluación

Cuestionario Clase 02

Calificación final

95 / 100

El profesor agregó comentarios a las preguntas de esta evaluación

Las puntuaciones de preguntas estarán disponibles después de que se hayan publicado todas las calificaciones

ENVIADO 13/4/26 19:50  
CONFIRMACIÓN: 30A571D319B34F1CB520E971C69B0436

95 / 100

1 VERDADERO/FALSO

HTML es un lenguaje de programación.

Verdadero

Falso

2 VERDADERO/FALSO

¿HTML se utiliza para crear y desarrollar páginas web?

Verdadero

Falso

3 VERDADERO/FALSO

¿Las agrupaciones de contenido en HTML no tienen ninguna relación con la forma en que los datos se ven o se comportan?

Verdadero

Falso

4 OPCIÓN MÚLTIPLE

¿En dónde se encuentra el contenido HTML?

A El contenido se encuentra entre la etiqueta de cierre y la etiqueta de apertura.

B El contenido se encuentra entre la etiqueta de apertura y la etiqueta de cierre.

C El contenido se encuentra antes de la etiqueta de apertura.

D El contenido se encuentra después de la etiqueta de cierre.

5 VERDADERO/FALSO

La **Etiqueta de cierre** es similar a la etiqueta de apertura con la diferencia que se encuentra entre < > antes de finalizar la etiqueta.

las acciones del usuario. Esta es la tecnología detrás de Ajax.

### Vínculos e imágenes

**Vínculo <a>**

```
<a href="https://ejemplo.com" target="_blank">Visitar sitio</a>
```

**href** URL del destino    **target="\_blank"** Abre en nueva pestaña

**Imagen <img> — void element**

```

```

**src** Ruta o URL de la imagen    **alt** Texto alternativo (accesibilidad, obligatorio)

⚠ **<img>** no tiene etiqueta de cierre — es un elemento vacío (void element)

La **Etiqueta de cierre** es similar a la etiqueta de apertura con la diferencia que se encuentra entre **<>** antes de finalizar la etiqueta.

☐ Verdadero

☒ Falso

6 VERDADERO/FALSO

¿HTML se utiliza junto con CSS y JavaScript para ampliar su funcionalidad?

☒ Verdadero

☐ Falso

7 VERDADERO/FALSO

JavaScript no permite añadir funciones avanzadas en una página web.

☐ Verdadero

☒ Falso

8 VERDADERO/FALSO

HTML5 es la quinta revisión y la versión más nueva del estándar HTML.

☒ Verdadero

☐ Falso

9 OPCIÓN MÚLTIPLE

¿Qué mejoras ofrece HTML5 en comparación con versiones anteriores de HTML?

☐ A Se limita a la creación de sitios web estáticos.

☒ B Soporte para la creación de aplicaciones web interactivas y conectadas a servidores de manera más efectiva.

☒ C Mejor soporte para medios enriquecidos.

☐ D HTML5 es igual a versiones anteriores de HTML.

**10 VERDADERO/FALSO**

Si el navegador no es compatible con HTML5 ignorará las nuevas características.

☒ Verdadero

☐ Falso

**11 VERDADERO/FALSO**

Declarar que el documento contiene marcado HTML5 con el tipo de documento HTML5 es muy simple:  
<!DOCTYPE html5>

☐ Verdadero

☒ Falso

**12 VERDADERO/FALSO**

No se especifica el conjunto de caracteres como primera medida en un documento HTML5

☐ Verdadero

☒ Falso

**13 OPCIÓN MÚLTIPLE**

¿Por qué es importante escribir un marcado HTML válido en HTML5?

- ☐ A Porque el marcado inválido solo se puede leer en navegadores antiguos.
- ☐ B Porque los navegadores compatibles con HTML5 no aceptan marcado inválido.
- ☒ C Porque el marcado válido es más fácil de leer y mantener.
- ☒ D Pero en el caso de errores en el marcado, todos los navegadores compatibles deben comportarse exactamente de la misma manera.
- ☐ E Porque el marcado válido aumenta la importancia de las incompatibilidades en varios navegadores más antiguos.

**14 OPCIÓN MÚLTIPLE**

¿Qué versión de HTML no existió?

- ☒ A HTML 1
- ☐ B HTML 4
- ☐ C HTML 2
- ☐ D HTML 3

**15 VERDADERO/FALSO**

La semántica en HTML no describe con mayor precisión cuál es su contenido.

- ☐ Verdadero
- ☒ Falso

**16 OPCIÓN MÚLTIPLE**

Las secciones de contenido en HTML5 son:

☐ A <place>

☒ B <header>

☒ C <aside>

☒ D <section>

**17 VERDADERO/FALSO**

<input> y <output> se utilizan en formularios.

☒ Verdadero

☐ Falso

**18 OPCIÓN MÚLTIPLE**

Algunos elementos nuevos de semántica son:

☒ A <figure>

☒ B <output>

☒ C <meter>

☒ D <mark>

☒ E <data>

19

VERDADERO/FALSO

^

MathML permite integrar directamente fórmulas matemáticas.

☒ Verdadero

☐ Falso

20

VERDADERO/FALSO

^

La conectividad permite comunicarse con el servidor.

☒ Verdadero

☐ Falso

21

OPCIÓN MÚLTIPLE

^

Web Sockets

☒ A Permite crear una conexión entre la página y el servidor.

☐ B Solo se permite HTML como formato en el intercambio de datos.

☐ C No establece una conexión permanente.

☒ D La conexión es permanente y se admite cualquier tipo de datos en el intercambio.



22

VERDADERO/FALSO



El evento de servidor enviados permite colocar eventos en un servidor.

☐ Verdadero

☒ Falso

23

VERDADERO/FALSO



La videoconferencia utiliza WebRTC

☒ Verdadero

☐ Falso

24

OPCIÓN MÚLTIPLE



Sin conexión y almacenamiento permite...

☐ A Almacenar datos remotamente en el lado del servidor.

☒ B Almacenar datos localmente en el lado del cliente.

☒ C Operar sin conexión.

25

VERDADERO/FALSO



Las aplicaciones pueden detectar si hay o no una conexión a Internet.

☒ Verdadero

☐ Falso

**26 VERDADERO/FALSO**

WHATWG restringe a las aplicaciones web almacenar datos estructurados en el lado del cliente.

☐ Verdadero

☒ Falso

**27 OPCIÓN MÚLTIPLE**

IndexedDB:

☒ A Es un estándar.

☒ B Permite búsquedas de alto rendimiento mediante índices.

☐ C Almacena grandes cantidades de datos no estructurados.

**28 VERDADERO/FALSO**

Para utilizar la Cámara Web debemos implementar complementos externos compatibles con HTML5

☐ Verdadero

☒ Falso

**29 VERDADERO/FALSO**

<canvas> permite dibujar gráficos y otros objetos.

☒ Verdadero

☐ Falso

**30 OPCIÓN MÚLTIPLE**

Web Workers

☐ A Ralentiza eventos interactivos.

☒ B Permite delegar la evaluación de JavaScript en segundo plano.

**31 VERDADERO/FALSO**

A pesar de su nombre, XMLHttpRequest puede ser usado para recibir cualquier tipo de dato, no solo XML, y admite otros formatos además de HTTP

☒ Verdadero

☐ Falso

**32 VERDADERO/FALSO**

El atributo wikiEditable transforma un sitio web en una wiki.

☐ Verdadero

☒ Falso

**33 OPCIÓN MÚLTIPLE**

Atributos de gestión del foco en HTML

☒ A hasFocus

☐ B activeTag

☒ C activeElement

☐ D requestAnimationFrame

**34 VERDADERO/FALSO**

Para bloquear el puntero al contenido utilizo API Pointer Lock

☒ Verdadero

☐ Falso

**35 VERDADERO/FALSO**

El navegador no puede detectar la orientación del dispositivo.

☐ Verdadero

☒ Falso

**36** OPCIÓN MÚLTIPLE

¿Cuál de las siguientes opciones representa la estructura mínima y correctamente formada de un documento HTML5?

☐ A `<!DOCTYPE html><html><head></head><body></body></html>`

☐ B `<html><head><title>Página</title></head><body></body></html>`

☐ C `!DOCTYPE html><html><head><title>Página</title></head><body></body></html>`

☒ D `<!DOCTYPE html><html><body><head><title>Página</title></head></body></html>`

**Comentarios**

Un documento HTML5 válido comienza con `<!DOCTYPE html>`, seguido de `<html>`, luego `<head>` (con al menos un `<title>`) y `<body>`. El orden `head → body` es obligatorio.

**37** VERDADERO/FALSO

Se puede registrar las aplicaciones web como controladores de protocolo.

☒ Verdadero

☐ Falso

**38 OPCIÓN MÚLTIPLE**

Fullscreen API

- ☒ A Controla el uso de la pantalla completa de una página Web.
- ☐ B Con la interfaz de usuario del explorador mostrada.
- ☒ C Sin la interfaz de usuario del explorador mostrada.

**39 OPCIÓN MÚLTIPLE**

Para construir una buena aplicación web offline-funcional

- ☐ A No es necesario saber cuando la aplicación esta realmente sin conexión.
- ☐ B No es necesario saber cuando sea devuelta a un estado con conexión.
- ☒ C Se necesita saber cuando la aplicación esta realmente sin conexión.
- ☒ D Es necesario saber cuando sea devuelta a un estado con conexión.

**40 VERDADERO/FALSO**

El uso de geolocalización permite a los navegadores localizar la posición del usuario mediante geolocalización.

- ☒ Verdadero
- ☐ Falso



41

## OPCIÓN MÚLTIPLE



¿Qué problema tiene el siguiente fragmento de HTML?

```
<p>Esto es un <strong>texto importante</p></strong>
```

- ☐ A El texto tiene un salto de línea inválido
- ☐ B No se puede usar `<strong>` dentro de `<p>`
- ☒ C Las etiquetas no están correctamente anidadas
- ☐ D Falta el atributo `lang` en `<html>`

42

## OPCIÓN MÚLTIPLE



¿Cuál es la diferencia principal entre los elementos `<head>` y `<body>`?

- ☒ A `<head>` contiene metadatos e información del documento (no visible); `<body>` contiene el contenido visible para el usuario
- ☐ B `<head>` define el estilo y `<body>` define la estructura
- ☐ C `<head>` es obligatorio solo en HTML5, `<body>` siempre fue obligatorio
- ☐ D No hay diferencia, ambos pueden contener texto visible

43

## OPCIÓN MÚLTIPLE



¿Cuál de los siguientes fragmentos está bien formado en HTML semántico para una lista de ítems?



DESARROLLO Y ARQUITECTURAS WEB

Facultad de Tecnología Informática

CLASE Nº 3

Profesor Ing. Sebastián Luna

[Sebastian.luna@uai.edu.ar](mailto:Sebastian.luna@uai.edu.ar)

UAI

Universidad Abierta Interamericana

AGENDA

HTML

Repaso  
Elementos  
Atributos

MARCADO DE TEXTO

Encabezados  
Párrafos  
Listas  
Vínculos

DEPURAR HTML

Código permisivo  
Validación HTML  
Interpretación de errores  
Conclusiones

Demo

HTML

- Repaso
- Elementos
- Atributos
- Anidar elementos
- Elementos vacíos

Introducción a las Rutas Absolutas y Relativas en HTML5

En el mundo del desarrollo web, la correcta gestión de las rutas de archivos y recursos es fundamental para la funcionalidad y la estructura de los sitios web. Una de las habilidades esenciales que todo desarrollador web debe dominar es el uso de rutas absolutas y relativas, especialmente al trabajar con elementos como `<img>` en HTML5.

Las rutas absolutas y relativas nos permiten especificar la ubicación de los recursos, como imágenes, hojas de estilo y scripts, en nuestros documentos HTML. Mientras que las rutas absolutas nos brindan una dirección específica y fija, las rutas relativas ofrecen flexibilidad y portabilidad, adaptándose a la estructura de directorios de nuestro proyecto.

En esta lección, exploraremos en detalle cómo y cuándo utilizar cada tipo de ruta, con ejemplos prácticos que ilustrarán su implementación en etiquetas `<img>`. Aprenderemos a navegar por la estructura de directorios de un sitio web, asegurándonos de que nuestros recursos se carguen correctamente, independientemente de dónde se encuentre el usuario o cómo se reestructure el sitio.

Prepárate para sumergirte en el mundo de las rutas en HTML5, un conocimiento crucial que te permitirá construir sitios web más robustos y mantenibles.

Rutas Absolutas

Una ruta absoluta especifica la ubicación exacta de un archivo o recurso en la web, incluyendo el protocolo (como http o https) y el dominio completo. Se utiliza cuando el recurso se encuentra en un sitio web diferente al del documento HTML.

Ejemplo de una ruta absoluta en una etiqueta `<img>`:

```

```

En este caso, no importa dónde se encuentre tu archivo HTML, siempre buscará la imagen en la dirección exacta que has proporcionado.

Rutas Relativas

Una ruta relativa hace referencia a un archivo en relación con la ubicación actual del documento HTML. Es útil cuando los recursos se encuentran en el mismo servidor y permite que los enlaces sigan funcionando incluso si se mueve toda la estructura de directorios a otra ubicación.

Ejemplo de una ruta relativa en una etiqueta `<img>`:

```

```

Si el documento HTML está en el mismo directorio que la carpeta *imagenes*, encontrará la imagen *logo.png* dentro de esa carpeta.

Rutas Relativas Avanzadas

- `./` indica el directorio actual.
- `../` se utiliza para subir un nivel en la estructura de directorios.

Ejemplo de uso de `../` en una etiqueta `<img>`:

```

```

Si tu archivo HTML está en un subdirectorio y necesitas acceder a la carpeta *imagenes* que está un nivel por encima, usarías `../` para subir un nivel.

Para bajar un nivel en la estructura de directorios en una ruta relativa, simplemente no necesitas agregar ningún prefijo especial. Por defecto, si proporcionas el nombre del archivo o carpeta, se asume que está en el mismo nivel que el documento HTML actual.

Por ejemplo, si tienes un archivo HTML en un directorio y quieres acceder a una imagen que está en un subdirectorio dentro de ese mismo nivel, usarías la ruta relativa directamente:

```

```

En este caso, subdirectorio es una carpeta que está en el mismo nivel que el archivo HTML, y *imagen.png* es el archivo de imagen dentro de esa carpeta.

Si la imagen está un nivel abajo del archivo HTML, usarías una ruta relativa que especifique el subdirectorio en el que se encuentra la imagen. Aquí tienes un ejemplo:

Supongamos que tu archivo HTML está en un directorio llamado *docs*, y la imagen está en un subdirectorio dentro de *docs* llamado *imagenes*. La estructura de directorios sería algo así:

```
docs/
├──
├── tuarchivo.html
├──
├── imagenes/
├── imagen.png
```

Para acceder a *imagen.png* desde *tuarchivo.html*, la ruta relativa sería simplemente el nombre del subdirectorio seguido del nombre del archivo de la imagen:

```

```

En este caso, *imagenes/* es el subdirectorio que está un nivel abajo del archivo HTML, y *imagen.png* es el archivo de imagen dentro de esa carpeta.

Es importante remarcar que las rutas relativas son más flexibles y facilitan la portabilidad del sitio web, mientras que las rutas absolutas son más específicas y más dependientes de la estructura del sitio web. Recuerda que las rutas relativas son útiles para mantener la portabilidad de tu sitio web, ya que permiten que los enlaces sigan funcionando incluso si cambias la ubicación de la carpeta del sitio web en el servidor.

## HTML



**HTML no es un lenguaje de programación; es un lenguaje de marcado que define la estructura de tu contenido.**

## HTML



**HTML consiste en una serie de elementos que usarás para encerrar diferentes partes del contenido para que se vean o comporten de una determinada manera.**

## HTML



Las etiquetas de encierre pueden hacer de una palabra o una imagen un hipervínculo a otro sitio, se pueden cambiar palabras a cursiva, agrandar o achicar la letra, etc.

## HTML



- Por ejemplo, observemos la siguiente línea de contenido:

```
Mi gato es muy gruñon
```

- Si queremos especificar que se trata de un párrafo, podríamos encerrar el texto con la etiqueta de párrafo (`<p>`):

```
<p>Mi gato es muy gruñon</p>
```

## HTML



Etiqueta de apertura

Etiqueta de cierre

```
<p>Mi gato es muy gruñon</p>
```

Contenido

Elemento

- **Etiqueta de apertura:** se reconoce porque está al inicio del código y se encuentra entre `<` `>` (paréntesis angulares). En este ejemplo son las que encierran la letra `p`.
- **Etiqueta de cierre:** son similares a la etiqueta de apertura con la diferencia que se encuentra una barra inclinada `</` antes de finalizar la etiqueta.
- **Contenido:** es la información que se encuentra entre la etiqueta de apertura y la etiqueta de cierre.

# HTML



Los elementos pueden también tener atributos, que se ven así:

Atributo

```
<p class="editor-note">Mi gato es muy gruñon</p>
```

## ATRIBUTOS



Los atributos contienen información adicional acerca del elemento, que no se desea que aparezca en el contenido real del elemento.

## ATRIBUTOS



Aquí **class** es el nombre del atributo y **editor-note** el valor del atributo.

## ATRIBUTOS



En este caso, el atributo **class** permite darle al elemento un nombre identificativo, que se puede utilizar luego para apuntarle al elemento información de estilo y demás cosas.

## ATRIBUTOS



### Un atributo debe tener siempre:

1. Un espacio entre este y el nombre del elemento (o del atributo previo, si el elemento ya posee uno o más atributos).
2. El nombre del atributo, seguido por un signo de igual (=).
3. Comillas de apertura y de cierre, encerrando el valor del atributo.

## ATRIBUTOS



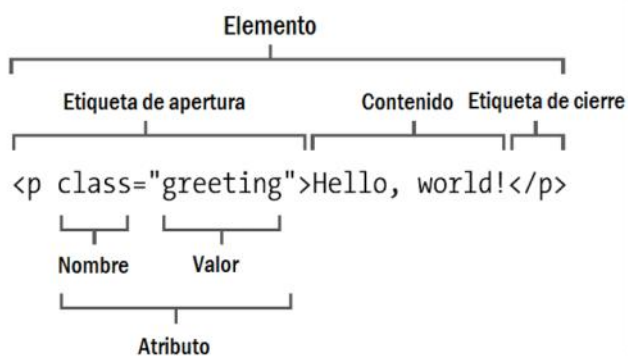
Los atributos siempre se incluyen en la etiqueta de apertura de un elemento, nunca en la de cierre.

## ATRIBUTOS



El atributo con valores simples que no contengan espacios en blanco ASCII (o cualesquiera de los caracteres " ' ` = < >) pueden permanecer sin entrecomillar, pero se recomienda entrecomillar todos los valores de atributo, ya que esto hace que el código sea más consistente y comprensible.

## HTML



## ANIDAR ELEMENTOS



Puedes también colocar elementos dentro de otros elementos —esto se llama anidamiento—.

## ANIDAR ELEMENTOS



Si se quiere resaltar una palabra del texto (en el ejemplo la palabra «muy»), podemos encerrarla en un elemento **<strong>**, que significa que dicha palabra se debe enfatizar:

```
<p>Mi gato es <strong>muy</strong> gruñon.</p>
```

## ANIDAR ELEMENTOS



Hay que asegurarse que los elementos estén correctamente anidados.

## ANIDAR ELEMENTOS



En el ejemplo de abajo, se creó la etiqueta de apertura del elemento **<p>** primero, luego la del elemento **<strong>**, por lo tanto, debes cerrar esta etiqueta primero, y luego la de **<p>**.



## ANIDAR ELEMENTOS



Esto sería incorrecto:

```
<p>Mi gato es <strong>muy gruñon.</p></strong>
```



## ANIDAR ELEMENTOS



Los elementos deben abrirse y cerrarse ordenadamente, de forma tal que se encuentren claramente dentro o fuera el uno del otro.



## ANIDAR ELEMENTOS



Si estos se encuentran solapados, el navegador web tratará de adivinar lo que se intenta decirle, pero puede que los resultados sean inesperados.



## ELEMENTOS VACÍOS



Algunos elementos no poseen contenido, y son llamados elementos vacíos.



## ELEMENTOS VACÍOS



Por ejemplo, el elemento `<img>` que sirve para incluir una imagen en el documento:

```

```



## ELEMENTOS VACÍOS



Posee dos atributos, pero no hay etiqueta de cierre `</img>` ni contenido encerrado.



## ELEMENTOS VACÍOS



Esto es porque un elemento de imagen no encierra contenido al cual afectar. Su propósito es desplegar una imagen en la página HTML, en el lugar en que aparece.



## ELEMENTOS VACÍOS



También se incluye un atributo **alt** (alternative) el cual contiene un texto que debería describir la imagen, y que podría ser accedido por usuarios que no pueden ver la imagen, quizás porque:



## ELEMENTOS VACÍOS



1) Son ciegos o tienen deficiencias visuales. Los usuarios con impedimentos visuales usualmente utilizan herramientas llamadas Lectores de pantalla (Screen Readers), los cuales les leen el texto contenido en el atributo **alt**.



## ELEMENTOS VACÍOS



2) Se produjo algún error en el código que impide que la imagen sea cargada. Como ejemplo, modifica deliberadamente la ubicación dentro del atributo **src** para que este sea incorrecto.



### MARCADO DE TEXTO

- Encabezados
- Párrafos
- Listas
- Vínculos

## ENCABEZADOS



Los elementos de encabezado permiten especificar que ciertas partes del contenido son encabezados, o sub-encabezados del contenido.



## ENCABEZADOS



De la misma forma que un libro tiene un título principal, y que a su vez puede tener títulos por cada capítulo individual, y subtítulos dentro de ellos, un documento HTML puede tenerlos también.

## ENCABEZADOS



HTML posee seis niveles de encabezados, <h1>—<h6>, aunque probablemente solo se llega a usar 3-4 como mucho:

```
<h1>Mi título principal</h1>  
<h2>Mi título de nivel superior</h2>  
<h3>Mi subtítulo</h3>  
<h4>Mi sub-subtítulo</h4>
```

## ENCABEZADOS



### ➤ Recomendaciones:

El encabezamiento de nivel 1 tiene un estilo implícito. No utilices elementos de encabezado para hacer el texto más grande o más oscuro, porque este elemento se utiliza por accesibilidad y otras razones como el posicionamiento en buscadores (Search Engine Optimization, SEO).

## ENCABEZADOS



### ➤ Recomendaciones:

Intenta crear una secuencia significativa de encabezados en tus páginas, sin saltarte niveles.



## PÁRRAFOS



Los elementos `<p>` se utilizan para encerrar párrafos de texto; se usa frecuentemente para el marcado de contenido de texto regular:

```
<p>Este es un simple párrafo</p>
```



## LISTAS



Mucho del contenido web está dado por listas, así que HTML tiene elementos especiales para ellas. El marcado de listas se realiza siempre en al menos dos elementos.



## LISTAS



Los dos tipos de listas más comunes son las listas ordenadas y las desordenadas.

## LISTAS



Las listas desordenadas son aquellas en las que el orden de los items no es relevante, como en una lista de compras. Estas son encerradas en un elemento `<ul>` (unordered list).

## LISTAS



Las listas ordenadas son aquellas en las que el orden sí es relevante, como en una receta. Estas son encerradas en un elemento `<ol>` (ordered list).

## LISTAS



Cada elemento de la lista se coloca dentro de un elemento **<li>** (list item).

```
<p>Las disciplinas más comunes son:</p>
```

```
<ul>
```

```
<li>Programación</li>
```

```
<li>Testing</li>
```

```
<li>Analistas</li>
```

```
</ul>
```

## VÍNCULOS



Los vínculos o enlaces son muy importantes —son los que hacen de la web, la web—.

## VÍNCULOS



Para implementar un vínculo, se utiliza un vínculo simple — **<a>** — la a es la abreviatura de la palabra inglesa «anchor» («ancla»).



## VÍNCULOS



Para convertir algún texto dentro de un párrafo en un vínculo, seguiremos estos pasos:



## VÍNCULOS



1. Encierra el texto en un elemento `<a>`, así:

```
<a>Manifiesto de Mozilla</a>
```



## VÍNCULOS



2. Proporcionaremos al elemento `<a>` un atributo `href`, así:

```
<a href="">Manifiesto de Mozilla</a>
```



## VÍNCULOS



**3. Completaremos el valor de este atributo con la dirección web con la que se quiera conectar al vínculo:**

```
<a href="https://www.mozilla.org/es-AR/about/manifiesto/">Manifiesto de Mozilla</a>
```

## VÍNCULOS



Se pueden obtener resultados inesperados si al comienzo de la dirección web se omite la parte `https://` o `http://` llamada protocolo.

## VÍNCULOS



Así que luego del marcado del vínculo, es conveniente hacer clic en él para asegurarse que se dirige a la dirección deseada.

# VÍNCULOS



## ➤ Consejo:

**href** podría parecer, en principio, una opción un tanto oscura para un nombre de atributo. Si tienes problemas para recordarla, recuerda que se refiere a *hypertext reference* (referencia de hipertexto).

## LISTAS DE ELEMENTOS



### EL ELEMENTO RAÍZ

| Elemento                  | Icono | Descripción          |
|---------------------------|-------|----------------------|
| <code>&lt;html&gt;</code> |       | Contenedor principal |

### METADATOS DEL DOCUMENTO

|                            |  |                                          |
|----------------------------|--|------------------------------------------|
| <code>&lt;head&gt;</code>  |  | Sección de encabezado del documento      |
| <code>&lt;title&gt;</code> |  | Título del documento                     |
| <code>&lt;base&gt;</code>  |  | URI base para resolver URIs relativos    |
| <code>&lt;link&gt;</code>  |  | Información relacional para el documento |
| <code>&lt;meta&gt;</code>  |  | Variable para el documento               |
| <code>&lt;style&gt;</code> |  | Atributos presentacionales               |

### SECCIONES

|                              |  |                                        |
|------------------------------|--|----------------------------------------|
| <code>&lt;body&gt;</code>    |  | El cuerpo del documento                |
| <code>&lt;article&gt;</code> |  | Contenido redistribuible               |
| <code>&lt;section&gt;</code> |  | Define una sección                     |
| <code>&lt;nav&gt;</code>     |  | Sección de navegación                  |
| <code>&lt;aside&gt;</code>   |  | Contenido sólo ligeramente relacionado |
| <code>&lt;h1&gt;</code>      |  | Encabezado de nivel 1                  |
| <code>&lt;h2&gt;</code>      |  | Encabezado de nivel 2                  |
| <code>&lt;h3&gt;</code>      |  | Encabezado de nivel 3                  |
| <code>&lt;h4&gt;</code>      |  | Encabezado de nivel 4                  |
| <code>&lt;h5&gt;</code>      |  | Encabezado de nivel 5                  |
| <code>&lt;h6&gt;</code>      |  | Encabezado de nivel 6                  |
| <code>&lt;hgroup&gt;</code>  |  | Agrupar encabezados consecutivos       |
| <code>&lt;header&gt;</code>  |  | El encabezado de una sección           |
| <code>&lt;footer&gt;</code>  |  | El pie de una sección                  |
| <code>&lt;address&gt;</code> |  | Información de contacto del autor      |

| AGRUPACIÓN DE CONTENIDO |                                                                                   |                                           |
|-------------------------|-----------------------------------------------------------------------------------|-------------------------------------------|
| <p>                     |  | Párrafo                                   |
| <hr>                    |  | Separador de contenido                    |
| <pre>                   |  | Bloque de texto preformateado             |
| <blockquote>            |  | Cita en forma de bloque                   |
| <ol>                    |  | Lista ordenada                            |
| <ul>                    |  | Lista no ordenada                         |
| <menu>                  |  | Inserta una barra de menú                 |
| <li>                    |  | Ítem de una lista                         |
| <dl>                    |  | Lista de descripciones                    |
| <dt>                    |  | Término en una lista de descripciones     |
| <dd>                    |  | Descripción en una lista de descripciones |
| <figure>                |  | Información autocontenida                 |
| <figcaption>            |  | Título para una figura                    |
| <main>                  |  | Contenido principal de una sección        |
| <div>                   |  | Contenedor genérico para bloques de texto |

| SEMÁNTICA DE TEXTO |                                                                                     |                                            |
|--------------------|-------------------------------------------------------------------------------------|--------------------------------------------|
| <a>                |    | Hypervínculo                               |
| <em>               |    | Texto con énfasis                          |
| <strong>           |    | Texto con énfasis fuerte                   |
| <small>            |    | Comentario secundario                      |
| <s>                |    | Contenido que ya no es preciso o relevante |
| <cite>             |  | Cita o referencia                          |
| <q>                |  | Cita en forma de línea                     |
| <dfn>              |  | Término definido en el texto circundante   |
| <abbr>             |  | Término abreviado                          |
| <ruby>             |  | Texto con anotaciones Ruby                 |
| <rt>               |  | Anotación Ruby                             |
| <rp>               |  | Texto a ser ignorado por anotaciones Ruby  |
| <data>             |  | Información apta para programas            |
| <time>             |  | Fecha y/u hora                             |
| <code>             |  | Código de computadora                      |

| SEMÁNTICA DE TEXTO |                                                                                     |                                            |
|--------------------|-------------------------------------------------------------------------------------|--------------------------------------------|
| <var>              |  | Instancia de una variable                  |
| <samp>             |  | Ejemplo de la salida de un programa        |
| <kbd>              |  | Texto ingresado por el usuario             |
| <sub>              |  | Texto en subíndice                         |
| <sup>              |  | Texto en superíndice                       |
| <div>              |  | Texto desplazado de la prosa normal        |
| <div>              |  | Texto desplazado del contenido circundante |
| <sup>              |  | Anotaciones no textuales                   |
| <mar>              |  | Marca texto en otro documento              |
| <bd>               |  | Aísla texto para el formato bidireccional  |
| <bd>               |  | Anula el algoritmo bidireccional           |
| <span>             |  | Contenedor genérico para líneas de texto   |
| <br>               |  | Quiebre de línea                           |
| <wbr>              |  | Oportunidad de quiebre de línea            |

| EDICIÓN              |                                                                                   |                                       |
|----------------------|-----------------------------------------------------------------------------------|---------------------------------------|
| <ins>                |  | Texto agregado                        |
| <del>                |  | Texto eliminado                       |
| CONTENIDO INCRUSTADO |                                                                                   |                                       |
| <picture>            |  | Imagen con múltiples orígenes         |
| <source>             |  | Recurso de medios alternativo         |
| <img>                |  | Imagen                                |
| <iframe>             |  | Contexto de navegación anidado        |
| <embed>              |  | Inserta aplicaciones externas         |
| <object>             |  | Inserta aplicaciones externas         |
| <param>              |  | Parámetro para una aplicación externa |
| <video>              |  | Video                                 |
| <audio>              |  | Audio                                 |
| <track>              |  | Pista de texto para videos            |
| <map>                |  | Mapa de imagen del lado cliente       |
| <area>               |  | Sector en un mapa de imagen           |

| LISTAS DE ELEMENTOS  |                                                                                     |                                   |
|----------------------|-------------------------------------------------------------------------------------|-----------------------------------|
| INFORMACIÓN TABULADA |                                                                                     |                                   |
| <table>              |    | Tabla                             |
| <caption>            |  | El título de una tabla            |
| <colgroup>           |  | Grupo de columnas                 |
| <col>                |  | Establece atributos para columnas |
| <tbody>              |  | Cuerpo de la tabla                |
| <thead>              |  | Encabezado de la tabla            |
| <tfoot>              |  | Pie de la tabla                   |
| <tr>                 |  | Fila                              |
| <td>                 |  | Celda común                       |
| <th>                 |  | Celda de encabezado               |

| FORMULARIOS |                                                                                     |                                    |
|-------------|-------------------------------------------------------------------------------------|------------------------------------|
| <form>      |  | Formulario                         |
| <label>     |  | Etiqueta de un control             |
| <input>     |  | Control de entrada                 |
| <button>    |  | Botón                              |
| <select>    |  | Lista de opciones                  |
| <datalist>  |  | Sugerencias para los controles     |
| <optgroup>  |  | Grupo de opciones en una lista     |
| <option>    |  | Una opción en una lista            |
| <textarea>  |  | Campo de texto multilinea          |
| <output>    |  | La salida de un proceso            |
| <progress>  |  | El progreso de una tarea           |
| <meter>     |  | Una medida                         |
| <fieldset>  |  | Grupo de controles                 |
| <legend>    |  | El título de un grupo de controles |



## LISTAS DE ELEMENTOS

### ELEMENTOS INTERACTIVOS

|           |                                                                                   |                                      |
|-----------|-----------------------------------------------------------------------------------|--------------------------------------|
| <details> |  | Contenido desplegable                |
| <summary> |  | Un resumen del contenido desplegable |
| <dialog>  |  | Cuadro de diálogo                    |

### PROGRAMAS

|            |                                                                                   |                                                |
|------------|-----------------------------------------------------------------------------------|------------------------------------------------|
| <script>   |  | Incrusta programas                             |
| <noscript> |  | Contenido alternativo para programas           |
| <template> |  | Plantilla para contenido a ser agregado        |
| <slot>     |  | Espacio en componentes para insertar contenido |
| <canvas>   |  | Contenedor para gráficos dinámicos             |

Para más detalles, visitar el siguiente sitio:

<https://www.htmlquick.com/es/reference/tags.html>

## DEPURAR HTML

- Consideraciones a tener en cuenta
- Código permisivo
- Validación HTML
- Interpretación de errores
- Conclusiones



## DEPURAR HTML

**Escribir HTML es fácil, pero ¿qué pasa si algo va mal y desconocemos dónde está el error de codificación?**

## DEPURAR HTML



HTML no se compila por separado antes de que el navegador lo analice (se interpreta, no se compila).

## DEPURAR HTML



Y la sintaxis de los elementos de HTML es mucho más sencilla de entender que la de cualquier lenguaje de programación real como C#, JavaScript o Python.

## DEPURAR HTML



La forma en que los navegadores ejecutan HTML es más permisiva que la de los otros lenguajes, cosa que es buena y mala a la vez.

## CÓDIGO PERMISIVO



¿Qué queremos decir con permisivo?

## CÓDIGO PERMISIVO



Normalmente cuando hacemos algo mal al codificar, suele haber dos tipos de error:

### CÓDIGO PERMISIVO

- Errores sintácticos:
  - Son errores de escritura en el código que impiden que el programa funcione.
  - Suelen ser fáciles de arreglar si estamos familiarizados con las herramientas adecuadas y sabemos el significado de los mensajes de error.





## CÓDIGO PERMISIVO

- Errores lógicos:
  - En estos errores la sintaxis es correcta, pero el código no hace lo que debería, por lo que el programa funciona de forma incorrecta.
  - Estos errores son, por lo general, más difíciles de solucionar que los sintácticos, porque no hay mensajes de error que nos avisen de ellos.

## CÓDIGO PERMISIVO

HTML en sí mismo no suele producir errores sintácticos porque los navegadores son permisivos con ellos; o sea, el código se sigue ejecutando.

## CÓDIGO PERMISIVO

Los navegadores disponen de reglas internas para saber cómo interpretar los errores de marcado incorrecto que encuentran, y seguirán funcionando aunque no produzcan el resultado esperado. Esto puede también ser un problema, por supuesto.

## CÓDIGO PERMISIVO



La ejecución de HTML es permisiva porque cuando se creó la web por primera vez, se decidió que el hecho de permitir que la gente publicara su contenido era más importante que el hecho de que la sintaxis fuera totalmente correcta.

## CÓDIGO PERMISIVO



La web no sería tan popular como lo es hoy en día si se hubiera sido más estricto desde el primer momento.

## VALIDACIÓN HTML



Podríamos asegurarnos de que nuestro HTML está bien formado.

## VALIDACIÓN HTML



**Pero... ¿qué pasa cuando trabajamos con documentos HTML grandes y complejos?**

VALIDACIÓN  
HTML

- La mejor estrategia es comenzar por pasar tu página HTML por el servicio de validación de etiquetas.
- <https://validator.w3.org/>

## VALIDACIÓN HTML



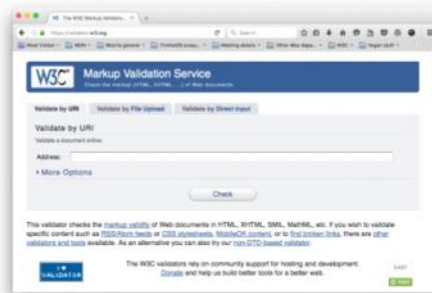
**Fue creado y está mantenido por el W3C, organización que se encarga de definir las especificaciones de HTML, CSS y otras tecnologías web.**

## VALIDACIÓN HTML



Esta página web toma un documento HTML como entrada, lo procesa, y genera un informe de dónde están los errores en el documento.

## VALIDACIÓN HTML



## VALIDACIÓN HTML



Para validar el HTML, podemos proporcionar al validador una dirección web a la que apuntar, subirle un archivo HTML, o directamente introducirle el código HTML que queremos que revise.

# VALIDACIÓN HTML



Vamos a probar de validar un documento ejemplo:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">

  <title>HTML debugging examples</title>

  <!--[if IE < 9]>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/html5shiv/3.7.3/html5shiv.js"></script>
  </endif-->
</head>

<body>
  <h1>HTML debugging examples</h1>

  <p>What causes errors in HTML?</p>

  <ul>
    <li><b>Unclosed elements</b>: If an element is <b>not</b> closed properly, then its effect can spread to areas you didn't intend.
    <li><b>Badly nested elements</b>: Nesting elements properly is also very important for code behaving correctly. <b>strong</b><b>strong</b><b>emphasised</b></strong> what is this?</li>
    <li><b>Unclosed attributes</b>: Another common source of HTML problems. Let's look at an example: <a href="https://www.mozilla.org">link to Mozilla homepage</a>.</li>
  </ul>
</body>
</html>
```

# VALIDACIÓN HTML



1. Primero, cargamos el servicio de validación en una pestaña del navegador, si no lo tenemos ya.
2. Hacemos clic en la subpestaña **Validate by Direct Input**.
3. Copiamos el código del documento ejemplo (no solo el body) y lo pegamos en el cuadro de texto grande.
4. Hacemos clic en el botón Check.

# VALIDACIÓN HTML



Esto debería proporcionar una lista de errores y otras informaciones:

1. **Info** The Content-Type was text/html. Using the HTML parser.
2. **Info** Using the schema for HTML with SVG 1.1, MathML 3.0, RDFa 1.1, and ITS 2.0 support.
3. **Error** End tag "li" implied, but there were open elements.  
From line 21, column 7, to line 21, column 10  
...nd<li>...</li>Badly
4. **Error** Unclosed element "strong".  
From line 19, column 47, to line 19, column 54  
lement is <strong>not cl
5. **Error** End tag "strong" violates nesting rules.  
From line 21, column 149, to line 21, column 157  
...phasized?</strong> what
6. **Error** End tag "li" implied, but there were open elements.  
From line 23, column 7, to line 23, column 10  
...a>...</li>Unclose
7. **Error** End of file reached when inside an attribute value. Invalid token

## INTERPRETACIÓN DE ERRORES



La lista de mensajes de error que nos presenta el validador suele ayudar, pero a veces, no resultan muy útiles; con un poco de práctica aprenderemos a interpretar la lista para arreglar nuestro código.

## INTERPRETACIÓN DE ERRORES



Veamos los mensajes de error y su significado. Observamos que cada mensaje se presenta con un número de línea y de columna, para ayudar a localizar el error más fácilmente.

## INTERPRETACIÓN DE ERRORES



- "Consider adding a lang attribute to the html start tag to declare the language of this document.": No se trata de un error, sino de una advertencia (warning). La recomendación es definir siempre un idioma, y este "warning" explica cómo hacerlo.

## INTERPRETACIÓN DE ERRORES



- "End tag li implied, but there were open elements" (2 instancias): Estos mensajes indican que un elemento que ha sido abierto debe ser cerrado. La etiqueta de cierre se supone, pero no está ahí. La información de la línea/columna apunta a la primera línea después de donde debería estar la etiqueta de cierre; es una buena pista para ver qué pasa.



## INTERPRETACIÓN DE ERRORES



- "Unclosed element strong": Un elemento `<strong>` no ha sido cerrado, y la línea/columna apunta directamente al lugar del error.



## INTERPRETACIÓN DE ERRORES



- "End tag strong violates nesting rules": Este apunta a elementos que están incorrectamente anidados, y la línea/columna nos indica donde están.





## INTERPRETACIÓN DE ERRORES



- "End of file reached when inside an attribute value. Ignoring tag": Esta es bastante enigmática; se refiere al hecho de que el valor de un atributo no ha sido bien construido, posiblemente cerca del final del archivo porque el final aparece dentro del valor del atributo. El hecho de que el navegador no muestre el enlace nos debería dar una buena pista de qué elemento es el erróneo.

## INTERPRETACIÓN DE ERRORES



- "End of file seen and there were open elements": Este es un poco ambiguo, pero básicamente se refiere al hecho de que hay elementos abiertos que necesitan ser cerrados adecuadamente. Los números de línea apuntan a las últimas líneas del archivo, y este mensaje de error viene con una línea de código que indica un ejemplo de un elemento abierto:

```
<a href="https://www.mozilla.org/>link to Mozilla homepage</a> </ul></body></html>
```

## INTERPRETACIÓN DE ERRORES



**Nota:** Un atributo al que le falten las comillas de cierre puede ser un elemento abierto, porque el resto del documento será interpretado como si fuera parte de este atributo.



## INTERPRETACIÓN DE ERRORES



Unclosed element ul: Este no ayuda mucho, porque el elemento `<ul>` está cerrado correctamente. Este error se debe a que el elemento `<a>` no ha sido cerrado, ya que faltan las comillas de cierre.

## CONCLUSIONES



No debemos preocuparnos si no podemos corregir todos los mensajes de error; es práctico tratar de arreglar unos pocos errores cada vez y volver a pasar el validador para ver los que quedan.

## CONCLUSIONES



A veces, al arreglar unos cuantos se arreglan automáticamente otros muchos; con frecuencia muchos errores los causa uno solo en un efecto dominó.

## CONCLUSIONES



Sabremos que todos los errores están arreglados cuando veamos el siguiente mensaje:

The document validates according to the specified schema(s) and to additional constraints checked by the validator.



¿Preguntas?

FIN



# DESARROLLO Y ARQUITECTURAS WEB

Facultad de Tecnología Informática

CLASE Nº4

Profesor Ing. Sebastián Luna

[Sebastian.luna@uai.edu.ar](mailto:Sebastian.luna@uai.edu.ar)

UAI

Universidad Abierta Interamericana

AGENDA

CSS

Introducción  
Sintaxis  
Anatomía  
Selectores

CONSIDERACIONES

body  
text-shadow  
Centrar una imagen

EJEMPLO DE CSS

Empezar con CSS  
Agregar CSS al HTML  
Dar formato al HTML

Demo

CSS

- Introducción
- Sintaxis
- Anatomía
- Selectores
- Relleno, marco y margen
- Color y cuerpo de la página

CSS

CSS (Hojas de Estilo en Cascada) es el código que se usa para dar estilo a una página web.

Introducción

## Cascading Style Sheets

Durante la clase, vamos a abordar material de lectura y audiovisual que nos permita comprender los siguientes temas:

- Utilización de CSS para aplicar estilos.
- Implementación de selectores.
- Capacidad de implementar diferentes atributos para darle estilos al HTML.
- Comprensión del concepto de Layout.

Para empezar, debemos comprender la importancia de un correcto uso de CSS:

Se trata de una tecnología utilizada para dotar de **calidades visuales y estéticas** a una página web.

Las siglas CSS (Cascading Style Sheets) significan «**hojas de estilo en cascada**» y parten de un concepto simple pero muy potente: **aplicar estilos** (colores, formas, márgenes, etc. ) a uno o varios documentos (generalmente documentos HTML, páginas webs) de forma masiva.

Se le denomina **estilos en cascada** porque se aplican de arriba a abajo (siguiendo un patrón denominado **herencia** que trataremos más adelante) y en el caso de existir ambigüedad, se siguen una serie de normas para resolverla. De forma predeterminada, los elementos de bloque comienzan en una **nueva** línea, pero los elementos en línea pueden comenzar en cualquier parte de una línea.

HTML

CSS

+

=

### La implementación de Selectores CSS:

Los selectores en CSS son el nexo de unión entre la hoja de estilos y los documentos a los que se aplique dicha hoja. Ya sean (x)HTML, XLM, SVG... y con independencia del medio en que que se muestre.

Del conocimiento y buen manejo que se haga del selector CSS dependerá la eficiencia de la hoja de estilos y en muchos casos el poder lograr lo deseado sin necesidad de recurrir a otros lenguajes.

El selector CSS es el **identificador** en la hoja de estilos del elemento o elementos del DOM a los que se aplicarán las declaraciones contenidas en la regla de la que forma parte dicho selector.

El término selector CSS puede referirse por igual a cualquiera de los cuatro tipo de selectores CSS que existen clasificados por la composición de su nombre:

- Un **selector css simple**
- Un **selector css compuesto**
- Un **selector css complejo**
- Una **lista de selectores css**

1.1. SELECTOR UNIVERSAL

1.2. SELECTOR DE TIPO ETIQUETA

1.3. SELECTOR DESCENDENTE

1.4. SELECTOR DE CLASE

1.5. SELECTOR ID

Que es Layout:

La palabra layout sirve para hacer referencia a la manera en que están distribuidos los elementos y las



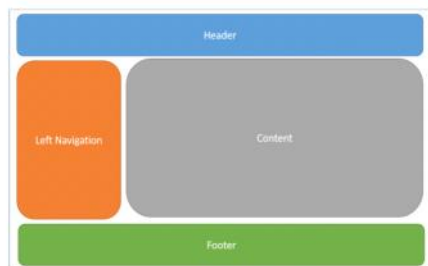
#### Que es Layout:

La palabra layout sirve para hacer referencia a la manera en que están distribuidos los elementos y las formas dentro de un diseño. Es un vocablo del idioma inglés que no forma parte del diccionario de la Real Academia Española. Se traduce al español como 'disposición', 'plan' o 'diseño'.

Se dice que el layout es la representación de un plano sobre el cual se va a dibujar la distribución de un espacio específico o determinado.

El layout puede tomarse como base para una página web, pues es a partir ese plan o diseño que esta se irá a desarrollar. Dicho de otro modo, el producto inicial de una página o sitio web es su layout, plantilla o diseño.

Por esto se dice que la plantilla de diseño que sirve para elaborar, predeterminar y desarrollar un sitio o página web se le llama layout. Este layout es un plan o esquema genérico, el cual es presentado por el diseñador al cliente para su aprobación, modificación o rechazo.



Lo invitamos a realizar la lectura del siguiente artículo, la cual servirá como complemento para comprender los conceptos más relevantes referidos a CSS y sus atributos:

• CSS Tutorial (<https://www.w3schools.com/css/default.asp>)



Video

#### Material Multimedia Complementario

Además, les recomendamos los siguientes videos que abordan con claridad cuestiones relacionadas con los conceptos de CSS, Atributos y Layout.

## CSS

CSS (Hojas de Estilo en Cascada) es el código que se usa para dar estilo a una página web.

## CSS

El enfoque de CSS es servir para definir la capa de presentación, es decir, la parte relacionada con el aspecto.

## CSS

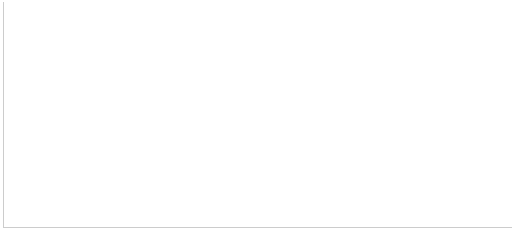
En el código **HTML** colocamos el contenido, es decir, qué debe visualizarse, mientras que con **CSS** definimos la presentación, es decir, cómo debe visualizarse.

## CSS

El CSS se puede usar para estilos de texto muy básicos como, por ejemplo, cambiar el color y el tamaño de los encabezados y los enlaces.

## CSS

Se puede utilizar para crear un diseño, como podría ser convertir una columna de texto en una



Se puede utilizar para crear un diseño, como podría ser convertir una columna de texto en una composición con un área de contenido principal y una barra lateral para información relacionada.

Incluso se puede usar para crear efectos de animación.

En CSS debemos seguir las buenas prácticas:

- ✓ Lo adecuado cuando trabajamos con **CSS** es escribir el código en archivos independientes, que tendrán extensión **.css**.

- ✓ No conviene colocar código **CSS** por tanto dentro de archivos **HTML**. Debemos evitar colocar estilos en etiquetas `<style>` dentro del propio código **HTML**.

- ✓ Por supuesto, mucho menos aconsejable es colocar estilos en los atributos "**style**" de las etiquetas **HTML**.

Es desarrollado por un grupo del W3C llamado CSS Working Group.

Este grupo está compuesto por representantes de proveedores de navegadores y otras compañías interesadas en CSS.

También hay otras personas, conocidas como expertos invitados, que actúan como voces independientes y no están vinculados a ninguna organización.

## CSS



El grupo desarrolla o especifica características nuevas del CSS.

## CSS



Algunas veces lo hacen porque un navegador en particular está interesado en alguna capacidad, otras porque los diseñadores y desarrolladores web piden una característica, y otras porque el grupo ha identificado un requisito.

## CSS



El CSS está en desarrollo constante y todos los días presenta nuevas características disponibles.

## SINTAXIS DE CSS



No es realmente un lenguaje de programación, tampoco es un lenguaje de marcado. Es un lenguaje de hojas de estilo, es decir, permite aplicar estilos de manera selectiva a elementos en documentos HTML.

## SINTAXIS DE CSS



Es un lenguaje basado en reglas: cada usuario define las reglas que especifican los grupos de estilos que van a aplicarse a elementos particulares o grupos de elementos de la página web.

## SINTAXIS DE CSS



Por ejemplo, para seleccionar todos los elementos de párrafo en una página HTML y volver el texto dentro de ellos de color rojo, se deberá escribir este CSS:

```
p {  
  color: red;  
}
```

## SINTAXIS DE CSS



Para proceder con la implementación del ejemplo, estas tres líneas de CSS se deberían escribir en un nuevo archivo de cualquier editor de texto y ser guardado como: style.css

## SINTAXIS DE CSS



Ahora solo resta aplicar el CSS al documento HTML:

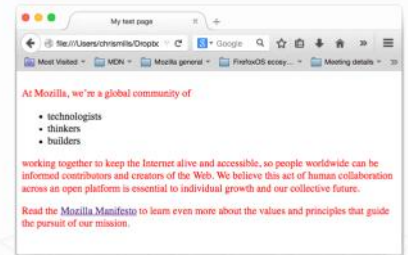
1. Abrimos el archivo **html** y escribimos la siguiente línea en algún lugar dentro del **<head>**, es decir, entre las etiquetas **<head>** y **</head>**:

```
<link href="styles/style.css" rel="stylesheet" type="text/css">
```



## SINTAXIS DE CSS

2. Guardamos el archivo **html** y lo abrimos en el navegador. Se debería ver algo como esto:



## ANATOMÍA DE UNA REGLA CSS



La estructura completa se denomina **regla predeterminada** (pero a menudo «regla» para abreviar).

- **Selector:** el (los) elemento **HTML** a dar estilo (en este caso, los elementos `<p>`).
- **Declaración:** especifica a cuál de las propiedades del elemento se quiere dar estilo.
- **Propiedades:** selección de la propiedad del elemento **HTML** que queremos afectar con la regla.
- **Valor de la propiedad:** a la derecha de la propiedad, después de los dos puntos (:), se escribe el valor de la propiedad.

## ANATOMÍA DE UNA REGLA CSS

Otras partes importantes de la sintaxis:

- Cada una de las reglas (aparte del selector) deben estar encapsuladas entre llaves (`{}`).
- Dentro de cada declaración, se debe usar los dos puntos (`:`) para separar la propiedad de su valor.
- Dentro de cada regla, se debe usar el punto y coma (`;`) para separar una declaración de la siguiente.

## ANATOMÍA DE UNA REGLA CSS

De este modo para modificar varios valores de propiedad a la vez, solo se necesita escribirlos separados por punto y coma (`;`)

```
p {  
  color: red;  
  width: 500px;  
  border: 1px solid black;  
}
```

## ANATOMÍA DE UNA REGLA CSS

También se puede seleccionar varios elementos y aplicar una sola regla a todos ellos. Incluye varios selectores separados por comas (,)

```
p,li,h1 {  
  color: red;  
}
```

## SELECTORES

Existen muchos tipos diferentes de selectores.

En el ejemplo solo se vio selectores de elementos, los cuales seleccionan todos los elementos de un tipo dado en los documentos [HTML](#).

## SELECTORES

Sin embargo se puede hacer selecciones más específicas que esas.

A continuación, se detallará algunos de los tipos de selectores más comunes:

## SELECTORES

| Nombre del selector                                                         | Qué selecciona                                  | Ejemplo                                 |
|-----------------------------------------------------------------------------|-------------------------------------------------|-----------------------------------------|
| Selector de elemento<br>(llamado algunas veces selector de etiqueta o tipo) | Todos los elementos HTML del tipo especificado. | <b>p</b><br>Selecciona <b>&lt;p&gt;</b> |

## SELECTORES

| Nombre del selector             | Qué selecciona                                                                                                       | Ejemplo                                                                                           |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Selector de identificación (ID) | El elemento en la página con el ID especificado (en una página HTML dada, solo se permite un único elemento por ID). | <b>#mi-id</b><br>Selecciona <code>&lt;p id="mi-id"&gt;</code> y <code>&lt;a id="mi-id"&gt;</code> |

## SELECTORES

| Nombre del selector | Qué selecciona                                                                                              | Ejemplo                                                                                                          |
|---------------------|-------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Selector de clase   | Los elementos en la página con la clase especificada (una clase puede aparecer varias veces en una página). | <b>.mi-clase</b><br>Selecciona <code>&lt;p class="mi-clase"&gt;</code> y <code>&lt;a class="mi-clase"&gt;</code> |

## SELECTORES

| Nombre del selector  | Qué selecciona                                            | Ejemplo                                                                                                   |
|----------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Selector de atributo | Los elementos en una página con el atributo especificado. | <b>img[src]</b><br>Selecciona <code>&lt;img src="mimagen.png"&gt;</code> pero no <code>&lt;img&gt;</code> |

## SELECTORES

| Nombre del selector     | Qué selecciona                                                                                                             | Ejemplo                                                                                                 |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Selector de pseudoclase | Los elementos especificados, pero solo cuando esté en el estado especificado, por ejemplo cuando el puntero esté sobre él. | <b>a:hover</b><br>Selecciona <code>&lt;a&gt;</code> , pero solo cuando el puntero esté sobre el enlace. |

# SELECTORES



Existen muchos más selectores para explorar:

|                                                                          |                   |                                                             |
|--------------------------------------------------------------------------|-------------------|-------------------------------------------------------------|
| <a href="#">Selector de tipo</a>                                         | h1 { }            | <a href="#">Los tipos de selectores</a>                     |
| <a href="#">Selector universal</a>                                       | * { }             | <a href="#">El selector universal</a>                       |
| <a href="#">Selector de clase</a>                                        | .box { }          | <a href="#">Los selectores de clase</a>                     |
| <a href="#">Selector de ID</a>                                           | #unique { }       | <a href="#">Los selectores de ID</a>                        |
| <a href="#">Selector de atributo</a>                                     | a[title] { }      | <a href="#">Los selectores de atributo</a>                  |
| <a href="#">Pseudoclase</a>                                              | p:first-child { } | <a href="#">Las pseudoclases</a>                            |
| <a href="#">Pseudoelemento</a>                                           | p::first-line { } | <a href="#">Los pseudoelementos</a>                         |
| <a href="#">Operadores de combinación descendentes</a>                   | article p         | <a href="#">Operadores de combinación descendentes</a>      |
| <a href="#">Operador de combinación de elementos hijo</a>                | article > p       | <a href="#">Operadores de combinación de elementos hijo</a> |
| <a href="#">Operador de combinación de elementos hermanos adyacentes</a> | h1 + p            | <a href="#">Hermanos adyacentes</a>                         |
| <a href="#">Operador de combinación general de elementos hermanos</a>    | h1 ~ p            | <a href="#">Hermanos generales</a>                          |

## RELLENO, MARCO Y MARGEN



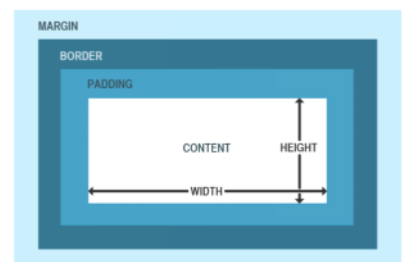
Una analogía que se puede utilizar sobre la escritura de CSS es que trata mucho sobre cajas —ajustando su tamaño, color, posición, etc.—.

## RELLENO, MARCO Y MARGEN



Se puede pensar en la mayoría de los elementos HTML de tu página como cajas apiladas una sobre la otra.

## RELLENO, MARCO Y MARGEN



## RELLENO, MARCO Y MARGEN

No es de extrañar que el diseño de CSS esté basado principalmente en el modelo de caja.

## RELLENO, MARCO Y MARGEN

Cada una de las cajas que ocupa espacio en la página tiene propiedades como estas:

## RELLENO, MARCO Y MARGEN

- **padding (relleno):** el espacio alrededor del contenido. En el ejemplo siguiente, es el espacio alrededor del texto del párrafo.
- **border (marco):** la línea que se encuentra fuera del relleno.
- **margin (margen):** el espacio fuera del elemento que lo separa de los demás.

RELLENO,  
MARCO Y  
MARGEN

margin

border

padding

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla id neque. Etiam vestibulum, augue sit amet condimentum imperdiet, diam neque blandit lacus, viverra ultrices nunc. Lorem laoreet orci. Integer tortor urna, viverra in, agnatis et, volutpat vel, nunc. Donec eget ipsum. Cras lacus. Nunc egetas ligula quis purus. Etiam viverra dignissim erat. Pellentesque facilisis. Quisque et eros eget ante condimentum.

## RELLENO, MARCO Y MARGEN

También se utilizan:

- **width**: ancho del elemento.
- **background-color**: color de fondo del contenido y del relleno.
- **color**: el color del contenido del elemento (generalmente texto).
- **text-shadow**: coloca una sombra difuminada en el texto dentro del elemento.
- **display**: selecciona el modo de visualización para el elemento.

## COLOR DE LA PÁGINA

Esta regla asigna un color de fondo a la página entera:

```
html {  
  background-color: #00539F;  
}
```

## CUERPO DE LA PÁGINA

Aplicar un estilo al cuerpo del documento:

```
body {  
  width: 600px;  
  margin: 0 auto;  
  background-color: #FF9500;  
  padding: 0 20px 20px 20px;  
  border: 5px solid black;  
}
```

## CUERPO DE LA PÁGINA

Explicación del estilo:

- **width: 600px;** - esto hará que el cuerpo siempre tenga 600 píxeles de ancho.
- **margin: 0 auto;** - cuando se selecciona dos valores dentro de propiedades como **margin** o **padding**, el primer valor afectará los lados superior (**top**) e inferior (**bottom**) (en este caso haciéndolo en 0), y el segundo valor los lados izquierdo (**left**) y derecho (**right**) (aquí, **auto** es un valor especial que divide el espacio disponible entre derecha e izquierda).

## CUERPO DE LA PÁGINA



Explicación del estilo:

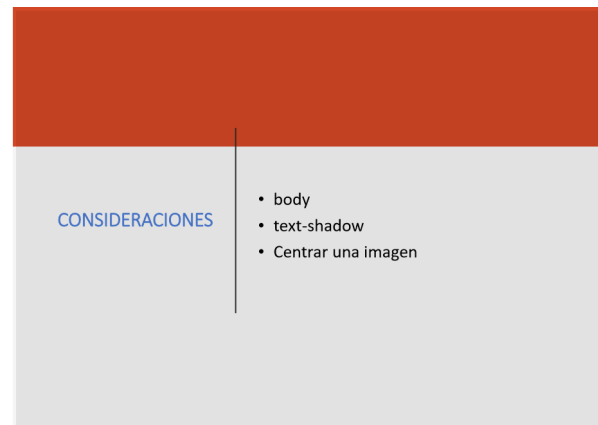
- **background-color: #FF9500;** - establece el color de fondo del elemento.
- **padding: 0 20px 20px 20px;** - hay 4 valores establecidos en el relleno, para dar un poco de espacio alrededor del contenido. No habrá relleno en la parte de arriba de **body**, 20 píxeles a la izquierda, abajo y derecha. Los valores se ponen: arriba, derecha, abajo e izquierda, en ese orden.

## CUERPO DE LA PÁGINA



Explicación del estilo:

- **border: 5px solid black;** - establece un borde de 5 píxeles de ancho, continuo y de color negro alrededor del elemento **body**.



## CONSIDERACIONES



Es posible que se note un hueco en la parte superior de **body**. Esto sucede porque los navegadores vienen con estilos por defecto...

<<Incluso cuando aún no se ha aplicado ningún archivo CSS>>



## CONSIDERACIONES



Una sugerencia para deshacerse de este espacio, hay que eliminar el estilo por defecto, agregando **margin: 0;**.

## CONSIDERACIONES



Una propiedad muy interesante es **text-shadow**, que aplica una sombra al texto del elemento.

## CONSIDERACIONES



- El primer valor en píxeles asigna el **desplazamiento horizontal** de la sombra desde el texto —qué tan lejos la mueve a la derecha—. Un valor negativo la moverá a la izquierda.
- El segundo valor en píxeles asigna el **desplazamiento vertical** de la sombra desde el texto —qué tan lejos la mueve hacia abajo—. En este ejemplo, un valor negativo la desplazaría hacia arriba.

## CONSIDERACIONES



- El tercer valor en píxeles asigna **radio de desenfoque** de la sombra —un valor grande es igual a una sombra borrosa—.
- El cuarto valor asigna el color base de la sombra.



## CONSIDERACIONES



Para centrar una imagen se puede usar nuevamente el truco de **margin: 0 auto** que se usa para **body**.

Existen algunas diferencias que se requieren para que el código CSS funcione.



## CONSIDERACIONES



El elemento **<body>** es un elemento en nivel de bloque (**block-level**), lo que significa que tomará espacio en la página y que puede tener otros valores de espacio aplicables como margen.



## CONSIDERACIONES



Las imágenes, por otra parte, son elementos **inline**, lo que quiere decir que no puedes aplicarles márgenes, debes dar a la imagen un comportamiento de **block-level** usando **display: block;**.



### EJEMPLO DE CSS

- Empezar con CSS
- Agregar CSS al HTML
- Dar formato al HTML

## EMPEZAR CON CSS



Nuestro punto de partida será un documento HTML, lo guardaremos localmente en nuestro equipo como `index.html`

## EMPEZAR CON CSS



```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>Empezamos con el CSS</title>
</head>
<body>

  <h1>Soy un título de nivel uno</h1>

  <p>Este es un párrafo de texto. En el texto hay un <span>elemento span</span> y también un <a
href="http://example.com">enlace</a>.</p>

  <p>Este es el segundo párrafo. Contiene un elemento <em>destacado</em>.</p>

  <ul>
    <li>Punto uno</li>
    <li>Punto dos</li>
    <li>Punto <em>tres</em></li>
  </ul>

</body>
</html>
```

## AGREGAR CSS AL HTML



Lo primero que se debe hacer es decirle al documento HTML que hay algunas reglas CSS que queremos que use.

## AGREGAR CSS AL HTML



Hay tres formas diferentes de aplicar **CSS** a un documento **HTML**, sin embargo, por ahora, veremos la forma más habitual y útil de hacerlo: vincular el CSS desde el encabezado del documento.

## AGREGAR CSS AL HTML



Creamos un archivo en la misma carpeta donde está documento **HTML** y lo guardaremos como **styles.css**.

La extensión **.css** muestra que es un archivo **CSS**.

## AGREGAR CSS AL HTML



Para vincular **styles.css** a **index.html**, añadimos la siguiente línea en algún lugar dentro del **<head>** del documento HTML:

```
<link rel="stylesheet" href="styles.css">
```

## AGREGAR CSS AL HTML



El elemento **<link>** le dice al navegador que hay una hoja de estilo con el atributo **rel** y la ubicación de esa hoja de estilo como el valor del atributo **href**.

## AGREGAR CSS AL HTML



Añadimos lo siguiente al archivo CSS:

```
h1 { color: red; }
```

## AGREGAR CSS AL HTML



Guardamos los archivos HTML y CSS antes de volver a cargar la página en un navegador web.

Ahora el título de nivel uno de la parte superior del documento debería ser rojo.

## DAR FORMATO AL HTML



Al poner nuestro título de encabezado en rojo, ya hemos demostrado que podemos elegir un elemento HTML y darle formato.

## DAR FORMATO AL HTML



Hacemos esto con un selector de elementos: un selector que coincide directamente con el nombre de un elemento HTML.

## DAR FORMATO AL HTML



Para determinar todos los párrafos del documento, se usa el selector `p`. Para hacer que todos los párrafos se vean verdes se usa:

```
p { color: green; }
```

## DAR FORMATO AL HTML



Podemos determinar múltiples selectores a la vez, separándolos con una coma. Si queremos que todos los párrafos y todos los elementos de la lista sean verdes, el código se verá así:

```
p, li { color: green; }
```



## WEBSITE LAYOUT

- Tipos
- Diseño efectivo
- Ejemplo de construcción

## WEBSITE LAYOUT



El diseño del sitio web (website layout) es una disposición de los elementos de la página destinados a proporcionar la mejor experiencia de usuario.



## WEBSITE LAYOUT



Determina la secuencia en la que los visitantes notan los componentes visuales y los objetos a los que prestan más atención.



## WEBSITE LAYOUT



Las primeras impresiones del sitio web están relacionadas con el diseño. Su diseño debe estar bien definido y ser atractivo para motivar a los visitantes a permanecer en él durante mucho tiempo.

## WEBSITE LAYOUT



- **Apariencia visual.** Los usuarios tardan unos 50 milisegundos en formarse una opinión sobre su sitio web. Su diseño debe ser intuitivo, simple y atractivo para atraer su atención. La gente necesita entender lo que vendes a la vez. Haga que cada página sea fácil de escanear y coloque los elementos de acuerdo con una jerarquía visual.

## WEBSITE LAYOUT



- **Navegación.** El 38% de los consumidores miran los enlaces de navegación por primera vez al visitar un sitio web. Las personas pueden navegar por el sitio web utilizando un menú de encabezado, un pie de página o una llamada a la acción. Es una buena idea agregar un botón "Volver al principio" y, de esa manera, hacer que la búsqueda sea más conveniente. Puede crear un mapa del sitio web para organizar todo el contenido de forma lógica.

## WEBSITE LAYOUT



- **Marca.** Se requiere un logotipo notable y el nombre de su empresa en un diseño bien definido. Su sitio web debe aumentar el conocimiento de la marca de los visitantes y permitirles distinguir fácilmente su negocio de los demás. Puede hacerlo implementando el mismo diseño en su empaque, redes sociales, sitio, ubicación fuera de línea e incluso en los productos. Además, elige una paleta de colores definida, ya que el 22% de los consumidores buscan colores llamativos cuando visitan una página web por primera vez.

## WEBSITE LAYOUT

- **Imágenes y videos.** Pueden transmitir tus valores, mostrar los productos que vendes o proporcionar información útil sobre tu marca. Esté extremadamente atento a la calidad del contenido, ya que el 40 % de los consumidores aprecian las fotos y las imágenes, y el 21 % cree que los videos son los principales elementos visuales en el diseño web.

## WEBSITE LAYOUT

- **Contenido.** Organice los datos usando un título, encabezados y listas. Es fundamental apegarse a un tono de voz, así que antes de escribir contenido para su sitio web, desarrolle un estilo que se alinearé con toda su marca. Proporcione solo datos útiles en un formato fácil de entender.

## WEBSITE LAYOUT

- **Tipografía.** Lo que escribes en tus textos es vital, pero cómo los escribes también es crucial. Elija una o dos fuentes legibles y utilícelas en todas sus cuentas de medios. Le permitirá aumentar la conciencia de marca y el reconocimiento de su negocio.

## WEBSITE LAYOUT

- **Diseño web y móvil.** Según Statista, los teléfonos móviles generan el 54,4% del tráfico mundial, por lo que es fundamental optimizar su sitio para usuarios de diferentes dispositivos. Recomendamos diseñar su sitio web de acuerdo con los principios de UX para brindar experiencias de usuario excepcionalmente positivas. Además, es fundamental desarrollarlo de acuerdo con los requisitos de SEO, para llegar a los primeros lugares en los resultados de búsqueda.

## WEBSITE LAYOUT

- **Optimización de velocidad.** Google afirma que la posibilidad de un rebote aumenta en un 32 % cuando el tiempo de carga de una página es de 1 a 3 segundos, y en un 90 % hasta los 5 segundos. Estas estadísticas demuestran que la optimización de la velocidad es fundamental. Para que su sitio web se cargue más rápido, debe comprimir imágenes, videos y gifs. Además, puede moverlo a un mejor host, reducir la cantidad de complementos y usar el almacenamiento en caché.

## WEBSITE LAYOUT

- **Espacio en blanco.** Ayuda a centrar la atención de los espectadores en los elementos más importantes. El espacio en blanco es necesario para crear textos más legibles, conectar diferentes componentes de un sitio web, desarrollar una jerarquía visual y hacer que el diseño sea más atractivo. Recomendamos dejar un espacio en blanco entre cada elemento de contenido para que sea más fácil de percibir.

## TIPOS DE WEBSITE LAYOUT

La elección del diseño más atractivo es fundamental.

## TIPOS DE WEBSITE LAYOUT

Permite hacer que el sitio web sea fácil de usar, centrar la atención de los visitantes en los elementos más importantes y transmitir el mensaje de marketing.



## TIPOS DE WEBSITE LAYOUT

A continuación, examinaremos los 10 tipos de diseño más populares y algunas ideas sobre cómo usarlos de manera efectiva.

### SINGLE COLUMN LAYOUT

- El diseño de una sola columna está muy extendido. Incluye el contenido en una columna vertical para que los espectadores puedan desplazarse para descubrir más datos. Este diseño se ve bien tanto en pantallas móviles como web. Los diseñadores a menudo lo usan para colocar publicaciones de blog de lectura prolongada. En tal caso, divide el texto en partes y agrega imágenes, videos y gifs para que el contenido sea más fácil de percibir.



### FULLSCREEN IMAGE LAYOUT

- El diseño de imagen de pantalla completa incluye solo una imagen y, a veces, un eslogan. Los espectadores no pueden desplazarse para encontrar más información, ya que se supone que su tipo representa los productos en lugar de describirlos. El diseño de imagen a pantalla completa le permite brindar una primera impresión sorprendente, ya que el 90 % de los datos que nuestro cerebro procesa son visuales. Sin embargo, no es adecuado para algunos sitios web, como blogs, revistas en línea, tiendas y otros en los que necesita colocar mucho contenido.



### HORIZONTAL STRIPE LAYOUT

- El diseño de franja horizontal incluye fotos, videos e ilustraciones que ocupan toda la pantalla. Cuando los usuarios se desplazan por dichos sitios web, ven nuevas imágenes a pantalla completa. Puede agregar esloganes pegajosos o descripciones breves para proporcionar más detalles. Este diseño le permite atraer a los consumidores y organizar la información en secciones mediante la creación de imágenes con varios colores de fondo.



## Z-PATTERN LAYOUT

- El diseño del patrón Z se basa en el comportamiento de escaneo de las personas. Los estudios dicen que los consumidores perciben pequeños fragmentos de información en forma de zigzag. Debes señalar los datos más importantes donde los espectadores dejan de concentrarse. Por ejemplo, los diseñadores suelen colocar logotipos en la esquina superior izquierda, menús de navegación en la parte superior y llamadas a la acción en la parte inferior de la página. Puede mezclar texto e imágenes para atraer a los consumidores y facilitar el escaneo.



## F-PATTERN LAYOUT

- El diseño del patrón F también se desarrolla de acuerdo con el comportamiento de escaneo de los espectadores. La gente lee grandes cantidades de datos en forma de 'F'. Se recomienda usar este diseño para publicaciones de lectura larga. Recuerda que los consumidores prestan más atención al texto del lado izquierdo, por lo que es mejor organizarlo mediante listas. Además, puede colocar un menú de navegación, pancartas u ofertas especiales arriba, ya que los consumidores siempre notan los elementos en la parte superior de la página.



The combination of great design and diligent app development.

[VIEW THEMES](#)



We make sleek and modern designs for your business.

## SPLIT SCREEN LAYOUT

- El diseño de pantalla dividida consta de dos columnas divididas verticalmente en el centro. Se adapta a las empresas que venden dos productos diferentes u ofrecen dos tipos diferentes de contenido. Además, puede usarlo si se dirige a dos segmentos de la audiencia, por ejemplo, hombres y mujeres, adolescentes y adultos, etc. El diseño de pantalla dividida le permite encontrar un equilibrio, pero puede parecer aburrido. Evite colocar demasiado texto, agregue imágenes y detalles animados para que su sitio web se vea más dinámico.

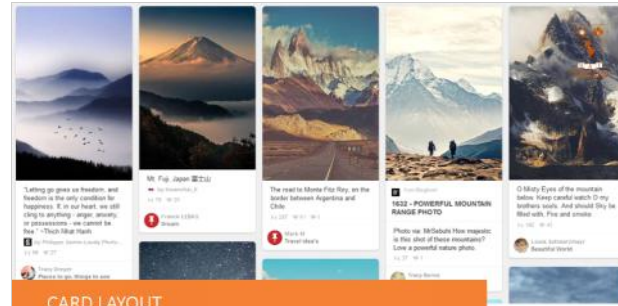


Keep everything organised without breaking your flow

Dropbox brings your files together, in one central place. They're easy to find and safely synced across all of your devices - so you can access them any time, anywhere. No more wasting the day tracking down work.

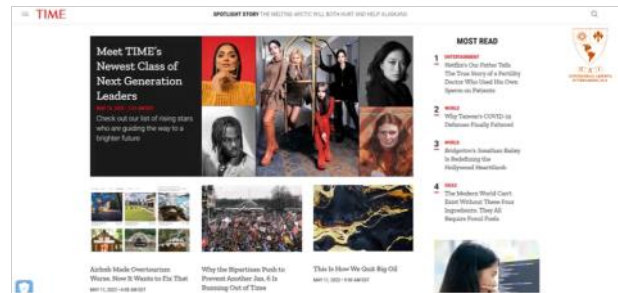
## ASYMMETRICAL LAYOUT

- El diseño asimétrico se basa en la pantalla dividida, pero las partes no son iguales. Le permite centrar la atención de los espectadores en las piezas de contenido más importantes al agregarles peso visual. Para ello, utiliza colores vivos y haz que ciertos elementos sean más grandes que otros. Un diseño asimétrico ayuda a que su diseño web sea más dinámico y atractivo y permita a las personas escanear páginas más rápido.



## CARD LAYOUT

- El diseño de la tarjeta consta de varias piezas de contenido organizadas en cuadros similares. Cada tarjeta generalmente incluye un título, una imagen y algunas oraciones de texto. Si los espectadores hacen clic en él, el navegador los lleva a una página separada que describe este producto. Este diseño se adapta a sitios web con mucho contenido. Para hacerlo más interactivo, puede agregar vistas previas del tamaño de un bocado. Preste atención al espacio en blanco entre las tarjetas, ya que la falta de este puede influir negativamente en la legibilidad de su sitio.



## MAGAZINE LAYOUT

- El diseño de la revista es necesario cuando se necesita colocar mucho contenido en una página. Se basa en una rejilla modular. Puede colocar contenido en las columnas y resaltar información vital agregando peso visual. Priorice los datos usando títulos y colores brillantes, agrande los contenidos más destacados y añádolos en el centro de la página. Su diseño debe permanecer legible y agregar espacios en blanco entre los elementos para que sea más fácil de escanear.

## DISEÑO EFECTIVO DE WEBSITE LAYOUT

La gente juzga la credibilidad de una empresa en función de su sitio web. La elección y el desarrollo de un diseño fácil de usar influye directamente en el número de visitantes.

## DISEÑO EFECTIVO DE WEBSITE LAYOUT

- Determine sus objetivos. El diseño del sitio web provoca que los espectadores realicen ciertas acciones, por lo que debe crearlo de acuerdo con los objetivos de su empresa. Es posible que desee aumentar el conocimiento de la marca de los consumidores, informarlos sobre algo o vender sus productos. Tenga en cuenta que los diseños pueden variar de una página a otra según sus objetivos.
- Estime la cantidad y la importancia del contenido que colocará. El tamaño del texto, las imágenes y los videos determina la elección de un diseño. Debe comprender en qué contenido necesita centrar la atención de los usuarios para crear una jerarquía visual y, al mismo tiempo, hacer que todos los elementos se vean holísticamente juntos.
- Crear wireframes. Te permitirá distribuir el espacio y priorizar los elementos esenciales. Presta especial atención al espacio en blanco y su distribución. El 79% de los consumidores solo escanean páginas, por lo que debes facilitar la percepción de los datos. Tenga en cuenta que su estructura debe adaptarse a todos los tamaños de pantalla, por lo que es fundamental comenzar desde el más pequeño, generalmente para teléfonos móviles, y agrandarlo gradualmente.
- Identificar estadísticas y crear contenidos. Luego debe elegir la paleta de colores y determinar la apariencia visual de su sitio web. El estilo debe permanecer igual en todos sus cuentas de medios y alinearse con su marca.
- Utilizar servicios de vanguardia para el desarrollo. Puede parecer difícil crear un sitio web, pero ahora hay muchos servicios que pueden ayudarlo. Puede insertar widgets de suscripción multicanal y agregar botones y múltiples enlaces a cuentas de redes sociales para optimizar su sitio web.
- Pruebe el rendimiento y optimícelo. Un diseño fácil de usar necesita mejoras constantes para mantenerse actualizado. Puede pedirle a algunas personas que prueben su sitio web y le den su opinión. Es mejor grabar en pantalla sus acciones para encontrar y solucionar los problemas de inmediato.

## EJEMPLOS DE WEBSITE LAYOUT

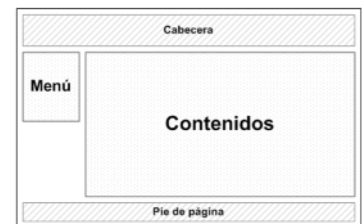
### Diseño a 2 columnas con cabecera y pie de página

El objetivo de este diseño es definir una estructura de página con cabecera y pie, un menú lateral de navegación y una zona de contenidos.



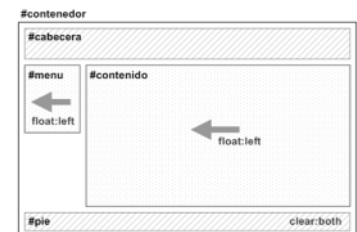
## EJEMPLOS DE WEBSITE LAYOUT

- La anchura de la página se fija en 700px, la anchura del menú es de 150px y la anchura de los contenidos es de 550px:



## EJEMPLOS DE WEBSITE LAYOUT

- La solución CSS se basa en el uso de la propiedad `float` para los elementos posicionados como el menú y los contenidos y el uso de la propiedad `clear` en el pie de página para evitar los solapamientos ocasionados por los elementos posicionados con `float`.



## EJEMPLOS DE WEBSITE LAYOUT

El código HTML y CSS mínimos para definir la estructura de la página sin aplicar ningún estilo adicional son los siguientes:



## EJEMPLOS DE WEBSITE LAYOUT

```
#contenedor {  
  width: 700px;  
}  
#cabecera {  
  menu {  
    float: left;  
    width: 150px;  
  }  
  #contenido {  
    float: left;  
    width: 550px;  
  }  
  #pie {  
    clear: both;  
  }  
}  
  
<body>  
<div id="contenedor">  
  <div id="cabecera">  
    </div>  
    <div id="menu">  
    </div>  
    <div id="contenido">  
    </div>  
    <div id="pie">  
    </div>  
  </div>  
</body>
```

## EJEMPLOS DE WEBSITE LAYOUT

En los estilos CSS anteriores se ha optado por desplazar tanto el menú como los contenidos hacia la izquierda de la página (**float: left**).

## EJEMPLOS DE WEBSITE LAYOUT

Sin embargo, en este caso también se podría desplazar el menú hacia la izquierda (**float:left**) y los contenidos hacia la derecha (**float: right**).

## EJEMPLOS DE WEBSITE LAYOUT

El diseño anterior es de anchura fija, lo que significa que no se adapta a la anchura de la ventana del navegador.

## EJEMPLOS DE WEBSITE LAYOUT



Para conseguir una página de anchura variable y que se adapte de forma dinámica a la ventana del navegador, se deben aplicar las siguientes reglas CSS:

## EJEMPLOS DE WEBSITE LAYOUT



```
#contenedor {  
}  
#cabecera {  
}  
#menu {  
  float: left;  
  width: 15%;  
}  
#contenido {  
  float: left;  
  width: 85%;  
}  
#pie {  
  clear: both;  
}
```

## EJEMPLOS DE WEBSITE LAYOUT



- Si se indican la anchuras de los bloques que forman la página en porcentajes, el diseño final es dinámico.
- Para crear diseños de anchura fija, basta con establecer las anchuras de los bloques en píxel.

